

Document OCR Pipeline

From Phone Photo to Searchable Text

"How do we turn a phone photo of a handwritten document into searchable, editable text?"

The Problem

Handwritten documents are difficult to digitize. Existing OCR tools work well on clean, printed text but struggle with handwriting — especially when the source image comes from a phone camera, which introduces distortion, uneven lighting, and orientation issues. There is no lightweight, end-to-end pipeline that takes a raw photo of a handwritten document and reliably extracts its text.

Camera Distortion

Phone photos introduce perspective warp, shadows, and uneven lighting that break standard OCR tools.

Handwriting Complexity

Printed OCR engines fail on cursive and mixed handwriting — the dominant format for personal documents.

No End-to-End Tool

Existing solutions require manual pre-processing steps or professional scanning equipment.

Goal: Build a lightweight pipeline that accepts a raw phone photo and returns clean, readable text — with no manual steps required.

OCR Approaches — Why TrOCR?

Approach	How It Works	Handwriting?	Phone Photos?	Verdict
Tesseract	Classic rule-based OCR (LSTM + heuristics)	Poor	Poor	✗ Not suitable
Google Vision API	Cloud-based CNN model, closed-source	Moderate	Good	⚠ Cloud dependency
EasyOCR	CNN encoder, open-source, pre-trained	Moderate	Moderate	⚠ Struggles with cursive
TrOCR (Microsoft)	Vision Transformer encoder + language model decoder	Excellent	Good*	✓ Best fit

**TrOCR requires clean input — hence the scan simulation step that precedes it in our pipeline.*

Why TrOCR Specifically

Architecture

Encoder: Vision Transformer (ViT) — splits the image into patches and learns spatial relationships between them, making it robust to varied handwriting styles.

Decoder: Transformer language model — generates text token by token, conditioned on the encoded image. This means it understands context, not just individual characters.

Pre-training: Trained on massive handwriting datasets (IAM, SROIE) — meaning it arrives with strong priors out of the box.

Key Advantages for This Project



Open-source

Available via HuggingFace — no API key, no cloud dependency, runs locally in Colab.



large model upgrade

Switching from trocr-base to trocr-large resolved key misreads (e.g. 'work' → 'more').



Line-level input

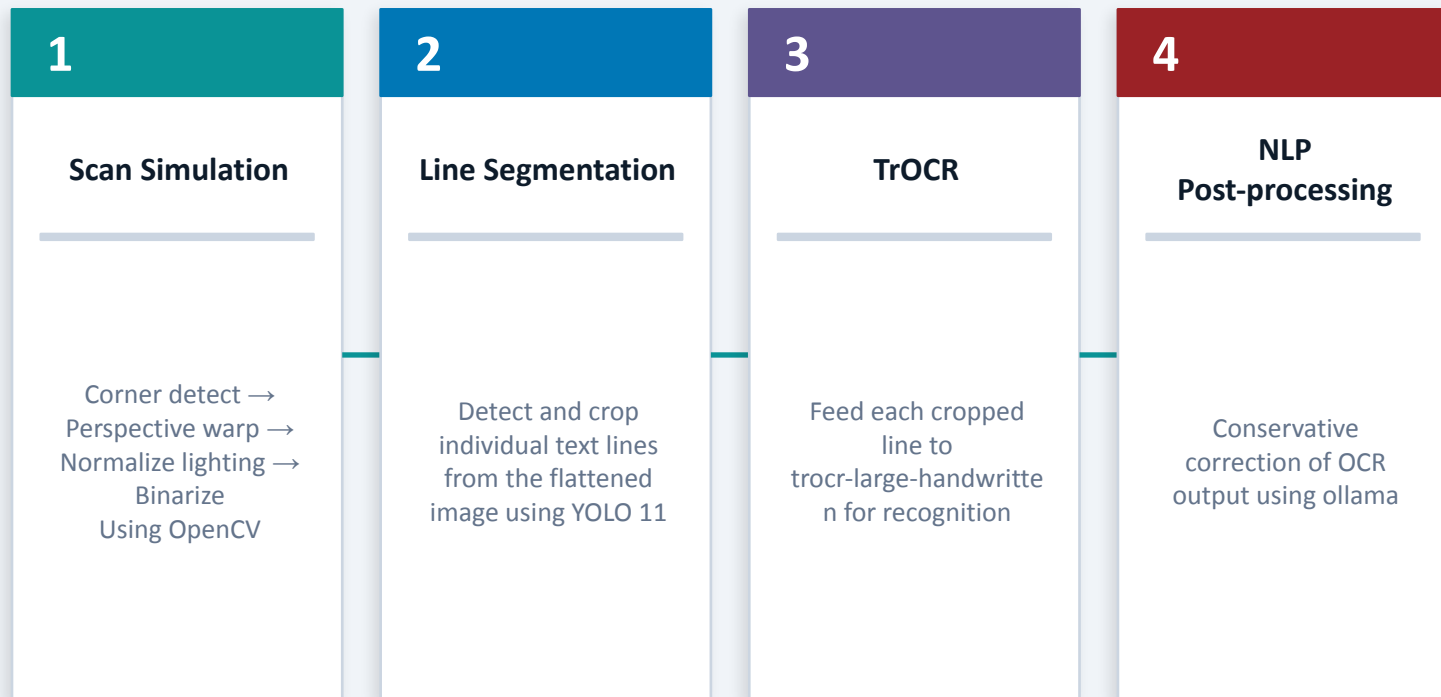
Designed to process one line at a time — aligns naturally with our line segmentation step.



Mixed text support

Handles both printed and handwritten text in the same document.

The Pipeline — 5 Steps



Input: Raw phone photo or scanner app output → Output: Clean, readable text

Step 1: Scan Simulation

The most critical step — transforms a noisy phone photo into a clean, scanner-like image before any OCR occurs.

1

Corner Detection

Canny edge detection → find contours → isolate the largest 4-point contour that represents the document boundary. Doubles as page detection.

OpenCV

2

Perspective Warp

Orders corners (TL→TR→BR→BL via sum/diff trick) → `getPerspectiveTransform` → `warpPerspective`. Corrects trapezoid distortion from camera angle.

OpenCV

3

Lighting Normalisation

`GaussianBlur(51,51)` estimates the background illumination. `cv2.divide` removes it, flattening uneven shadows across the page.

OpenCV

4

Adaptive Binarisation

`adaptiveThreshold` with block size 51, constant 15. Converts to clean black-and-white, suppressing background noise while preserving ink strokes.

Tuned

Step 2: Line Segmentation

Detects and crops individual text lines from the flattened document — powered by a fine-tuned YOLO11 model.

1

Dataset — Roboflow

Training data sourced from Roboflow: annotated images of handwritten document lines with bounding box labels. Roboflow handled augmentation, splitting, and export in YOLO format.

Roboflow

2

Model — YOLO11

Fine-tuned YOLO11 on the Roboflow dataset. YOLO11 detects all text line bounding boxes in a single forward pass — fast and accurate across varying line heights and spacings.

YOLO11

3

Inference & Cropping

Model runs on the scan-simulated image and returns bounding boxes sorted top-to-bottom. Each detected region is cropped and passed individually to TrOCR — one crop per inference call.

OpenCV

4

Why Not Contour-Based?

Classical contour or projection-based line detection breaks on slanted lines and variable spacing. YOLO11 handles these naturally because it has learned from real handwriting examples.

Design

Step 3: TrOCR Integration

Each cropped line from Step 2 is fed individually to microsoft/trocr-large-handwritten for text recognition.

1

Model Loading

TrOCRProcessor and VisionEncoderDecoderModel loaded from HuggingFace (microsoft/trocr-large-handwritten). The processor handles image resizing and pixel normalisation to the format ViT expects.

HF Hub

2

Image Preparation

Each cropped line (NumPy array from OpenCV) is converted directly to a PIL RGB image — no disk I/O. RGB conversion is a format requirement only; the image remains visually black-and-white from binarisation.

Pillow

3

Inference

processor(image, return_tensors='pt') tokenises the image into patches. model.generate() runs the ViT encoder then the transformer decoder to produce token IDs, which processor.batch_decode() converts to a text string.

TrOCR

4

Model Size Decision

Upgraded from trocr-base to trocr-large after base produced consistent misreads on test images (e.g. 'work' recognised as 'more'). The large model resolved these at the cost of higher memory and slower inference.

Tuned



Current Limitations & Next Steps

Known Limitations

— Perspective correction fails without a clear document boundary

— Can't handle angle 90 rotations with Exif metadata

— Paper must be rectangular in shape, no torn pieces or odd shapes

— Can't handle mathematical equations and diagrams, strictly text

Thank you